

Accelerated Fourier SAT: Fully Realising a GPU-based Pseudo-Boolean SAT Solver

Cody J. Christopher ✉ 

School of Computing, Australian National University, Canberra, Australia.

Charles Gretton ✉ 

School of Computing, Australian National University, Canberra, Australia.

Abstract

We present Accelerated Fourier SAT (AFSAT), a GPU-accelerated solver for pseudo-Boolean satisfiability based on continuous local search (CLS). AFSAT realises the proof-of-concept approach, FastFourierSAT, into a fully-engineered solver supporting any heterogeneous mixture of symmetric constraint types and lengths within a single problem instance. Using the JAX compiler, AFSAT leverages pure function composition, automatic vectorisation, automatic differentiation, and just-in-time (JIT) compilation to perform massively parallel CLS across batches of candidate assignments. We demonstrate substantially improved numerical stability, runtime performance, and memory efficiency over the proof-of-concept. We achieve this by way of identifying and addressing various limitations that arise from memory latency and floating-point representation, as well as leveraging automatic parallelisation and compact representations. The inherent representational and stability limitations of floating point are partially addressed by a tailored discrete Fourier transform implementation. We achieve near-linear throughput when scaling to multiple accelerators via JAX array sharding.

2012 ACM Subject Classification Mathematics of computing → Combinatorial optimization; Mathematics of computing → Solvers; Mathematics of computing → Nonconvex optimization

Keywords and phrases Satisfiability, pseudo-Boolean, SAT Solver, continuous local search, combinatorial optimization, hardware acceleration

Digital Object Identifier [10.4230/LIPIcs...](https://doi.org/10.4230/LIPIcs...)

Supplementary Material AFSAT implementation with Apache-2.0/GPL-2.0-or-later licenses:

Software (Source Code): <https://github.com/cjchristopher/accelerated-fourier-sat>

1 Introduction

Continuous local search (CLS) offers a compelling search paradigm for solving satisfiability (SAT) problems that are expressed using symmetric pseudo-Boolean (PB) constraints. The approach relaxes Boolean problem variables to real-valued variables by way of the Walsh-Fourier transform from Boolean function analysis [7]. Revisiting this approach, SAT is reformulated as a bounded continuous optimisation problem amenable to gradient-based search methods. This approach was formalised for the FourierSAT proof-of-concept [6] and subsequently extended for parallelised GPU computation in FastFourierSAT [3], which demonstrated that the Walsh-Fourier expansion can be evaluated efficiently using a vectorised Discrete Fourier Transform (DFT) in $O^*(\log k)$ time where k is the number of variables (typically also literals) in a constraint (clause), and $O^*(\cdot) \stackrel{\text{def}}{=} O_{p \rightarrow \infty}(T_p(\cdot))$ is idealised parallel execution time with infinite resources.

We present Accelerated Fourier SAT (AFSAT), a ground-up re-engineered and extended implementation of this CLS approach. AFSAT provides the following improvements and novel contributions as a tool:

- **Heterogeneous constraint support.** AFSAT is the first CLS implementation to support problems with an arbitrary mix of common PB constraint types of varying lengths within a single problem instance.



© Cody J Christopher and Charles Gretton;
licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

- 45 ■ **Improved performance and efficiency.** We demonstrate better execution times (up
46 to parity in the worst case) with less overhead, substantially reduced GPU memory
47 consumption, and higher peak throughput (evaluations/searches per unit time) compared
48 to `FastFourierSAT`.
- 49 ■ **Multi-GPU scaling.** We demonstrate near-linear scaling across multiple GPUs using
50 distributed array sharding in JAX [5], in a single-program-multiple-data (SPMD), or
51 compute-follows-data paradigm.
- 52 ■ **Numerical stability and precision improvements.** We identify and address sources
53 of floating-point errors and representational deviations through a tailored DFT matrix
54 construction with deferred division. This establishes a practical maximum constraint
55 length of approximately 50 variables.
- 56 ■ **Partial assignment integration.** AFSAT accepts partial variable assignments as input,
57 opening up avenues to use it as a sub-solver within decomposition-based architectures
58 such as DAGSTER [2], or other portfolio approaches.

59 A companion paper [4] presents the theoretical analysis of the representational advantages
60 of native pseudo-Boolean formulations, and novel empirical findings arising from this work
61 relating to the dynamics of CLS.

62 2 Background

63 2.1 Continuous Relaxation via Walsh-Fourier Expansion

64 CLS operates on a continuous relaxation of a SAT problem. For a Boolean formula ϕ
65 over n variables, we seek a relaxation as a polynomial on the Boolean hypercube $\mathcal{Q}^n$ that
66 preserves satisfiability (i.e. a solution to the relaxed formulation provides a solution for the
67 discrete problem). The (Walsh-)Fourier expansion (FE) [7] of Boolean functions provides this
68 relaxation, mapping $\{\text{True}, \text{False}\}$ to $\{-1, 1\}$:

$$69 \quad \text{FE}_\phi(\mathbf{X}) \stackrel{\text{def}}{=} \sum_{S \in 2^{\mathbf{X}}} \hat{f}(S) \prod_{x_i \in S} x_i \quad (1)$$

70 where $\hat{f}(S)$ are called the Fourier coefficients. For *symmetric* constraints—those whose
71 truth value depends only on the *simple count* of true literals—closed-form solutions for
72 the coefficients exist that are computable in polynomial time at worst [6, 4]. Symmetric
73 constraints necessarily have idempotent variable weights.

74 2.1.1 Satisfaction as Optimisation

75 Given a formula ϕ in n variables, decomposed into m symmetric constraints $C_1, \dots, C_m$, the
76 satisfiability problem is expressed as the bounded minimisation:

$$77 \quad \min_{\mathbf{X}} \sum_{k=1}^m \text{FE}_{C_k}(\mathbf{X}) \text{ subject to } \mathbf{X} \in \mathcal{Q}^n \quad (2)$$

78 where an assignment $\mathbf{X} \in [-1, 1]^n$ *satisfies* ϕ if $\sum_k \text{FE}_{C_k}(\mathbf{X}) = -m$. This formulation is
79 differentiable, non-convex, bounded, and saddle-dense. This motivates the use of projected
80 and/or bounded gradient search methods as the first choice of search algorithms. Observing
81 that FE is differentiable and fast parallelised automatic differentiation is available, we would
82 also consider second- or higher-order methods, should they exist.

2.1.2 DFT-Based Vectorised Evaluation

A key insight from Cen et al. [3] is that the evaluation of the Fourier expansion can be performed using a DFT. We observe that the terms in $\mathbf{FE}_\phi$ sharing a coefficient $\hat{f}(S)$ are all the size $|S|$ combinations of the variables of the clause. These particular sums are well-studied polynomials known as the *elementary symmetric polynomials* (ESPs). Since the sequence of all length k ESPs up to n variables, e_k^n , can be computed with (0-padded) linear convolutions, we can compute the evaluation of a Fourier expanded formula with vectorised operations in the frequency domain by making use of the linear convolution theorem:

$$\begin{aligned}
 \hat{f}_\phi &\equiv [\hat{f}_\phi(\emptyset), \hat{f}_\phi(1), \dots, \hat{f}_\phi(n)] \\
 \mathbf{e}^n &\equiv [e_0^n, e_1^n, \dots, e_n^n] = ([x_1, 1, 0, \dots] * [x_2, 1, 0, \dots] * \dots * [x_n, 1, 0, \dots]) \\
 &= W^* W ([x_1, 1, 0, \dots] * [x_2, 1, 0, \dots] * \dots * [x_n, 1, 0, \dots]) \\
 &= W^* (W ([x_1, 1, 0, \dots]) \cdot W ([x_2, 1, 0, \dots]) \cdot \dots \cdot W ([x_n, 1, 0, \dots])) \\
 &= W^* ([1 + x_1, \omega + x_1, \dots, \omega^n + x_1] \cdot \dots \cdot [1 + x_n, \dots, \omega^n + x_n]) \\
 &= W^* \left([1, \omega, \omega^2, \dots, \omega^n]^T + [x_1, x_2, \dots, x_n] \right) \quad \triangleright (\text{Outer addition}) \\
 &= W^* \left(\left[\prod_{i=1}^n (1 + x_i), \prod_{i=1}^n (\omega + x_i), \dots, \prod_{i=1}^n (\omega^n + x_i) \right] \right) \\
 &= W^* \left(\left[\prod_{i=1}^n (\omega^j + x_i) \right]_{j=0}^n \right) \\
 \mathbf{FE}_\phi &\equiv \sum (\hat{f}_\phi \cdot \mathbf{e}^n) = \sum \left(\hat{f}_\phi W^* \cdot \left(\left[\prod_{i=1}^n (\omega^j + x_i) \right]_{j=0}^n \right) \right)
 \end{aligned}$$

Where $*$ is the convolution operator on sequences, $\cdot$ is the Hadamard product (element-wise multiplication), W is the DFT matrix in $n+1$ dimensions (W^* the conjugate transpose), and ω the primitive $(n+1)^{\text{th}}$ root of unity. By taking the DFT of the ESP sequence convolution, the computation can be expressed as element-wise products and sums.

3 System Design and Implementation

3.1 Architecture

AFSAT is implemented in Python using the JAX [5] and XLA compilers. The architecture comprises:

- (a) **Problem ingestion:** The specification and parsing of PB-encoded problem instances specified in standard DIMACS or our hybrid variant (Appendix A).
- (b) **Fourier coefficient computation:** Closed-form computation of coefficients for all supported symmetric constraint types (§3.2).
- (c) **DFT precision:** Tailored calculation of roots of unity for conjugate symmetry (§3.5.1).
- (d) **Compiled solver kernel:** JIT-compiled search algorithm with automatic differentiation, vectorised over batches of candidate assignments.
- (e) **Multi-GPU distribution:** Sharding of batched assignments across available GPUs.

The solver kernel is compiled once and executed repeatedly with different random initialisations. Compilation produces an optimised XLA HLO program targeting the available accelerator architecture. AFSAT exposes many additional parameters and heuristic options which can be adjusted to target specific problem types.

120 3.2 Supported Constraint Types

121 Unlike prior CLS implementations which are restricted to either a single constraint type, or a
 122 particular fixed set of constraint types per problem, AFSAT supports heterogeneous problems
 123 containing any combination of the following symmetric pseudo-Boolean constraint types:

Constraint Type	PB Form	Coefficient Cost
Disjunction (OR)	$\sum_n x_n \geq 1$	$O(1)$
At most one (AMO)	$\sum_n x_n \leq 1$	$O(n)$
Exactly one (EO)	$\sum_n x_n = 1$	$O(n)$
Exactly k (EK)	$\sum_n x_n = k$	$O(n \log^2 n)$
Not all equal (NAE)	$\bigwedge \begin{cases} \sum_n x_n < n \\ \sum_n x_n > 0 \end{cases}$	$O(1)$
Exclusive Or (XOR)	$\sum_n x_n \equiv 1 \pmod{2}$	$O(n)$
Cardinality- k (CARD)	$\sum_n x_n \geq k$	$O(n \log^2 n)$

■ **Table 1** Supported symmetric PB constraint types and asymptotic cost of computing their Walsh-Fourier expansion coefficients ($\hat{f}(S)$).

124 This heterogeneous support enables AFSAT to handle problems with native PB formulations
 125 directly, avoiding the representational blowup of CNF translation.

126 3.3 Gradient Descent and other search Algorithms

127 We follow FastFourierSAT and select Projected Gradient Descent (PGD, Algorithm 1) as
 128 a fast baseline search algorithm for AFSAT. We also provide various other algorithms that
 129 possess compatible implementations. For any selected search algorithm, we take a vector-map
 130 across a batch of B candidate assignments. This is realised by JAX in GPU warps, whereby
 131 every candidate assignment has dedicated memory and a streaming processor, and the chosen
 132 algorithm executes in lockstep across every candidate in the batch. For the choice of PGD,
 133 each descent operates numerically independently: a line search determines the step size, a
 134 gradient step is taken, and the result is projected back onto the bounded subspace $\mathcal{Q}^n$ if we
 135 happen to step out. Termination occurs when either a convergence criterion is met (e.g. new
 136 location is within ϵ of the previous location) or given step-limit is reached. The convergence
 137 criteria is checked after every step, and some candidates in the batch may converge sooner
 138 than others. As parallelism is achieved via warps, the converged candidates execute the
 139 equivalent of no-ops until the entire batch finishes.

140 3.4 Partial Assignment Support

141 AFSAT accepts a partial variable assignment as input, fixing specified variables across all
 142 batched starting valuations while randomising the remainder. JAX supports gradient masking
 143 (zeroing) to prevent updates to variables flagged as fixed during gradient computation by
 144 fixing the corresponding gradient to 0. This feature enables integration into decomposition
 145 frameworks where a partial assignment from systematic search is completed by CLS, where
 146 some orchestrator may track candidate assignments for solvers in a portfolio to improve

■ **Algorithm 1** Projected Gradient Descent

Input: Fourier expanded formula FE_ϕ , initial valuation $\mathbf{X}_{(0)}$, maximum iterations d , convergence threshold δ , maximum step-size s , bounds $\mathcal{Q}$

Output: Finishing assignment $\bar{\mathbf{X}}$, evaluation $\text{FE}_\phi(\bar{\mathbf{X}})$, unsatisfied clause count $\#(\sum_m 1 \cdot \text{FE}_{g_m}(\bar{\mathbf{X}}) > 0)$, iterations t

```

for  $t \leftarrow 1$  to  $d$  do
   $\eta \leftarrow \text{lineSearch}(\text{FE}_\phi, \nabla(\text{FE}_\phi), s);$  // Determine step length
   $\mathbf{X}_{(t)} \leftarrow \mathbf{X}_{(t-1)} - \eta \cdot \nabla(\text{FE}_\phi(\mathbf{X}_{(t-1)})) ;$  // Take descent step
   $\mathbf{X}_{(t)} \leftarrow \text{projectToBounds}(\mathbf{X}_{(t)}, \mathcal{Q}) ;$  // Return to bounds
   $\text{is\_sat} \leftarrow \text{True}$  if  $\text{unsat\_count}(\phi, \mathbf{X}_{(t)}) = 0$  else  $\text{False};$ 
  if  $\eta < \delta$  or  $\text{is\_sat}$  then // Converged or SAT
    return  $\mathbf{X}_{(t)}, \text{FE}_\phi(\mathbf{X}_{(t)}), \text{unsat\_count}(\phi, \mathbf{X}_{(t)}), t;$ 
  end
end
return  $\mathbf{X}_{(t)}, \text{FE}_\phi(\mathbf{X}_{(t)}), \text{unsat\_count}(\phi, \mathbf{X}_{(t)}), d;$ 

```

147 or advise upon. AFSAT evenly divides up a given batch of candidate assignment amongst
 148 multiple partial assignment if they are provided. During clause processing, AFSAT will also
 149 detect trivial unit literals and either fold them into all provided partial assignments, or
 150 convert them to a universal partial assignment

151 3.5 Floating-Point Limits on Constraint Length

152 The DFT-based evaluation for a constraint of length n involves products $\prod_{i=1}^n (\omega^j + x_i)$
 153 which, for roots $\omega^j \approx 1$ and literal values $x_i \approx 1$, can reach magnitudes of 2^n , while for
 154 $\omega^j \approx -1$ these products approach values on the order of ϵ^n where $|\epsilon| \rightarrow 0$. For IEEE-754
 155 64-bit arithmetic with machine epsilon $\epsilon = 2^{-52}$, the dynamic range of these terms exceeds
 156 representable precision when $n \gtrapprox 50$, causing catastrophic cancellation in the inverse DFT
 157 and producing evaluations $|\text{FE}_\phi| \gg 1$.

158 We empirically observe degenerate solver behaviour—exploding and vanishing gradients—
 159 for constraints of length $n \geq 48$ for which many of variables trend toward the same truth
 160 value (e.g. all but one assignment in an AMO constraint of length 50 will be false (1)). This
 161 establishes a practical ceiling for CLS on current GPU architectures that lack practical
 162 support for extended-precision arithmetic.

163 3.5.1 Tailored DFT Construction

164 Standard DFT implementations (such as those from common scientific processing packages
 165 e.g., from SCIPY [9]) introduce cumulative errors through repeated exponentiation of roots
 166 of unity, breaking the conjugate symmetry required for precise cancellation in the inverse
 167 DFT. Since AFSAT’s algebraic foundation is sensitive to these inaccuracies, we implement or
 168 own tailored procedures:

- 169 ■ Forward and inverse DFT matrices are constructed to guarantee exact conjugate symmetry
 170 between paired terms.
- 171 ■ Combinatorial terms for Fourier coefficients are computed using arbitrary-precision integer
 172 arithmetic, with division and conversion to float deferred to the latest possible stage.
- 173 ■ Where algebraic symmetries exist, terms are explicitly mirrored rather than independently
 174 computed.

4 Performance Evaluation

All experiments were conducted on the GADI supercomputer (NCI, Australia), using nodes from the Volta GPU partition. Each node provides four NVIDIA Tesla V100 GPUs (32GB HBM2 each, $\approx 900\text{GB/s}$ bandwidth), Intel Xeon Cascade Lake CPUs (48 cores), and $\approx 192\text{GB}$ system RAM.

4.1 Comparison with FastFourierSAT

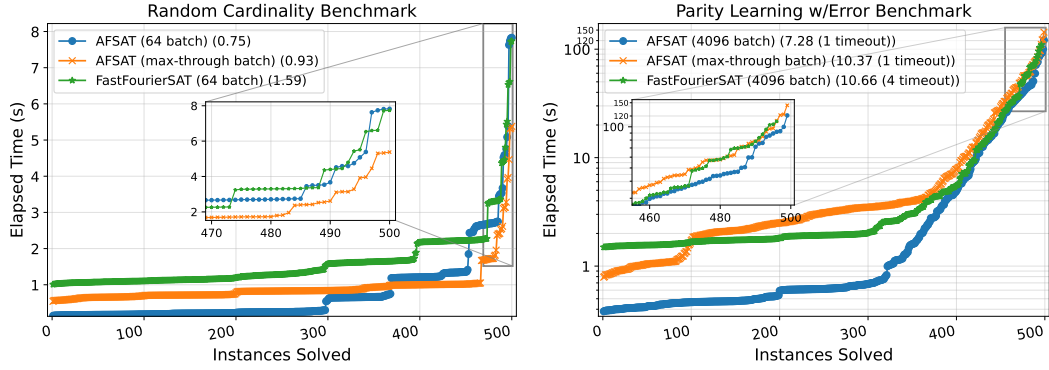


Figure 1 Replicated benchmarks for random cardinality (*Subfig. a*) and parity learning (xor) with errors (*Subfig. b*) as per `FastFourierSAT` [3], with PAR-2 scores and timeouts indicated. We run AFSAT configured as close to possible to `FastFourierSAT`, replicating the batch size choice as indicated in the original benchmark. We also run AFSAT in *max-through* mode, where the batch size is selected to target maximum efficiency (see §4.2).

We replicate the cardinality constraint and parity-learning benchmarks from Cen et al. [3]. Figure 1 compares cumulative solution times. We apply certain JAX optimisations set for AFSAT to `FastFourierSAT` to provide a more level playing field, and note that the parity learning results shown for `FastFourierSAT` are better than those presented in their paper due to bugs in the published code we have corrected. Key observations:

- AFSAT achieves equivalent performance in the worst-case, and otherwise universally improves upon `FastFourierSAT`, demonstrating that our optimisations strictly improved upon the proof-of-concept on comparable benchmarks.
- AFSAT exhibits an obviously lower baseline times than `FastFourierSAT`, indicating more efficient pre-processing, GPU kernel optimisation/compilation, and general overheads. For fairness, we take the best time of several runs for each to allow for disk latency and compilation caches to be populated.
- AFSAT consumes substantially less GPU memory by storing only minimal auxiliary data (e.g. highly optimised DFT matrices, clause and literal arrays, etc.) and computing closures over this data, enabling the compilers to better optimise which subsequently enables larger batch sizes.

4.2 GPU Memory and Throughput Characteristics

During testing it was noticed that as total GPU memory utilisation increased (reflecting larger batch sizes), the total number of completed gradient descents decreased both relatively (with respect to the amount of memory consumed) and in some cases absolutely. We investigate the total number of completed searches we can take per unit time, a measure we call *throughput*,

as a function of GPU memory consumption and GPUs available across several problem domains (Figure 2). We observe clearly that throughput peaks when the memory used is approximately 0.1%–1% of total GPU memory consumed. When exploring this effect across various GPUs, we find this percentage varies, but upon closer inspection the value is more tightly correlated twice the cache memory available on the device. We observe a logarithmic decay (Figure 2b) in throughput beyond this peak, rather than the monotonic increases one might expect when parallelising a problem up to memory saturation.

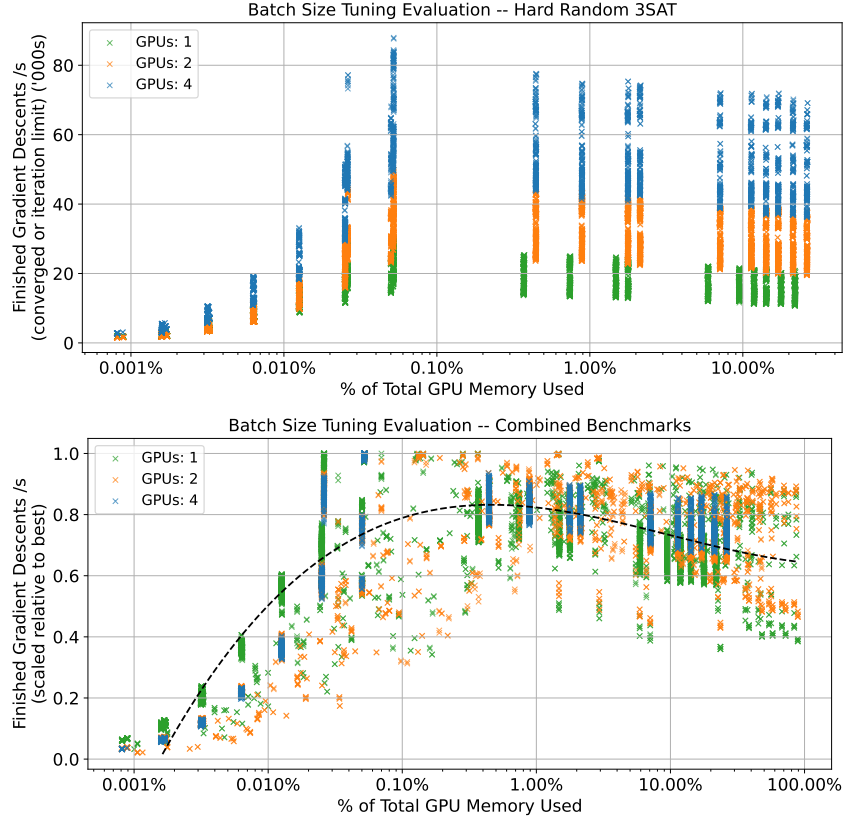


Figure 2 Throughput vs measured memory consumption across various problem classes. (Subfig. a) shows near linear scaling when parallelising across several accelerators on various hard random 3SAT problems, with the occasional exception due to properties of individual test cases. (Subfig. b) shows scaled throughput for various benchmarking problems, confirming the dynamics of throughput vs. batch size (memory consumption). A polynomial trendline peaks between 0.1% and 1%.

Across various domains we also observe the optimal batch size (and subsequently peak throughput) is governed primarily by maximum constraint length rather than the absolute number of variables or constraints. This is dominated ultimately by the requirement to compute $O(k^2)$ DFT matrices for k length clauses. As the size of the DFT grows quadratically it rapidly cuts the amount of memory usable for the remainder of the algorithm, resulting in lower batch sizes. We hypothesise that the throughput peak corresponds to saturation of low-level processing core caches rather than main memory, whereby optimal pipelining is achieved with next to no cache invalidation or page-faulting. Supporting this, the ratio of total cache to total memory on the V100 architecture is on the order of 0.1%—consistent with the observed peak locations.

This finding has practical implications: for problems with long constraints, smaller batch sizes yield higher throughput than larger ones that exceed cache capacity. For deployment, AFSAT provides batch-size tuning that balances throughput against total search coverage. As indicated in §3.5, long constraints also degrade numerical stability. The combination of both of these effects suggests that future improvements will look to trade-off the compactness of single constraints for the efficiency gains of constraint decompositions.

JAX’s sharding mechanism distributes batched valuations across available GPUs in a compute-follows-data paradigm. Since gradient descents are independent across each assignment in the batch, communication overhead is minimal. We observe near-linear scaling in throughput with increasing GPU count across all tested problem domains and Figure 2a demonstrates this for hard random 3SAT problems. The absence of an impacting overhead cost confirms the suitability of CLS for multi-accelerator deployment.

5 Conclusion and Future Work

AFSAT advances the state of CLS-based SAT solving from proof-of-concept to a practical, extensible tool. By engineering support for heterogeneous pseudo-Boolean constraints, achieving substantial performance improvements over the baseline, and demonstrating scalable multi-GPU execution, AFSAT establishes a foundation for GPU-accelerated SAT solving in accelerator-rich environments. The identification of precision-driven constraint-length limits and cache-driven throughput characteristics provides actionable guidance for deployment. Future development will target adaptive constraint weighting, integration with systematic solvers via decomposition frameworks, and exploitation of emerging higher-precision accelerator arithmetic.

Constraint length ceiling and memory consumption.

As established in §3.5 and §4.2, floating-point precision limits constraint length to $n \approx 50$ on 64-bit hardware and dominates memory consumption. Problems with naturally longer constraints (e.g., global cardinality constraints in large graph colouring) can be addressed by decomposing these into shorter equivalent constraints. For all but EK and CARD, there are simple linear decompositions to $O(k/50)$ equivalent PB constraints of the same type, preserving some of the benefits of native PB representation.

Incompleteness.

AFSAT is an incomplete solver: it cannot prove unsatisfiability, as doing so would require solving the global optimisation problem to certifiable optimality. However, AFSAT functions naturally as a MaxSAT solver, providing best-effort solutions with unsatisfied-constraint counts. Unbounded global optimisation methods are a potential consideration, and methods involving Moreau envelopes [3] or unconstrained formulations with extreme penalties [10].

Second-order methods.

The multi-linear objective is thoroughly populated with saddle points, at which first-order PGD can stall. Bounded second-order methods (e.g., L-BFGS-B) would help escape saddle points, but existing implementations in the JAX ecosystem—notably JAXOPT [1] and OPTIMISTIX [8]—do not correctly respect bounds or are not production-ready. Higher order methods will necessarily decrease throughput due to memory required to compute and store higher order gradients.

References

- 1 Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-López, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and modular implicit differentiation. *arXiv preprint arXiv:2105.15183*, 2021.
- 2 Mark Alexander Burgess, Charles Gretton, Josh Milthorpe, Luke Croak, Thomas Willingham, and Alwen Tiu. Dagster: Parallel structured search with case studies. In Sankalp Khanna, Jian Cao, Quan Bai, and Guandong Xu, editors, *PRICAI 2022: Trends in Artificial Intelligence*, pages 75–89, Cham, 2022. Springer Nature Switzerland.
- 3 Yunuo Cen, Zhiwei Zhang, and Xuanyao Fong. Massively parallel continuous local search for hybrid sat solving on gpus. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(11):11140–11149, 2025.
- 4 Cody Christopher and Charles Gretton. A study of parallel continuous local search. Companion paper, 2025.
- 5 DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Antoine Dedieu, Claudio Fantacci, Jonathan Godwin, Chris Jones, Ross Hemsley, Tom Hennigan, Matteo Hessel, Shaobo Hou, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Markus Kunesch, Lena Martens, Hamza Merzic, Vladimir Mikulik, Tamara Norman, George Papamakarios, John Quan, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Laurent Sartran, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Miloš Stanojević, Wojciech Stokowiec, Luyu Wang, Guangyao Zhou, and Fabio Viola. The DeepMind JAX Ecosystem, 2020. URL: <http://github.com/google-deeppmind>.
- 6 Anastasios Kyrillidis, Anshumali Shrivastava, Moshe Y Vardi, and Zhiwei Zhang. Solving hybrid boolean constraints in continuous space via multilinear fourier expansions. *Artificial Intelligence*, 299:103559, 2021.
- 7 Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014.
- 8 Jason Rader, Terry Lyons, and Patrick Kidger. Optimistix: modular optimisation in jax and equinox. *arXiv:2402.09983*, 2024.
- 9 Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.
- 10 Zhiwei Zhang, Samy Wu Fung, Anastasios Kyrillidis, Stanley Osher, and Moshe Y. Vardi. Thinking out of the box: Hybrid sat solving by unconstrained continuous optimization, 2025. URL: <https://arxiv.org/abs/2506.00674>, arXiv:2506.00674.

300 **A** Input Format Specification

301 AFSAT accepts problem instances in standard DIMACS CNF or an extended DIMACS-like
302 hybrid format supporting heterogeneous symmetric pseudo-Boolean constraints.

303 DIMACS-like PB constraint grammar

304 The hybrid PB problem grammar is defined as follows, where tokens are whitespace-separated:

```

<file>      ::= <line>*
<line>      ::= <comment> | <problem> | <constraint>
<comment>   ::= ('c' | '*') <text>
<problem>   ::= 'p' 'cnf' <posint> <posint>
<constraint> ::= [<prefix>] <body> '0'
<prefix>    ::= 'h'
<body>      ::= <cnf> | <typed> | <counted>
<cnf>       ::= <lit>+
<typed>     ::= <type_id> <lit>+
<type_id>   ::= 'x' | 'xor' | 'n' | 'nae'
              | 'a' | 'amo' | 'e' | 'eo'
<counted>   ::= <count_id> <lit>+
<count_id>  ::= <exactly> | <card>
<exactly>   ::= ('k' | 'ek') <int>
<card>      ::= ('d' | 'card') <ineq_op> <posint>
<ineq_op>   ::= '<' | '<=' | '>' | '>='
<lit>       ::= <int> \ {0}
<int>       ::= '-'? [1-9] [0-9]*
<posint>    ::= [1-9] [0-9]*

```

305 The type identifiers map to constraint semantics as follows:

Identifiers	Type	Semantics
(none)	CNF	Standard disjunction
x, xor	XOR	Odd number of literals true
n, nae	NAE	Not all literals equal
a, amo	AMO	At most one literal true
e, eo	EO	Exactly one literal true
k, ek	EK	Exactly k literals true
d, card	CARD	Cardinality threshold

■ **Table 2** Constraint type identifiers and semantics.

306 For CARD constraints with a plain integer threshold (no operator): positive k defaults to $\geq k$;
307 negative k defaults to $< |k|$. All CARD constraints are internally normalised to $\geq$ or $<$ forms.

308 Examples

```

c --- Standard DIMACS CNF ---
p cnf 4 2
1 -2 3 0
-1 4 0

c --- Hybrid format (mixed constraints) ---
p cnf 6 7
h 1 -2 3 0          c CNF: (x1 OR NOT x2 OR x3)
h x 1 2 3 0         c XOR: odd parity over {x1,x2,x3}
h nae 1 2 3 0        c NAE: not all of {x1,x2,x3} equal (at least one true & false)
h amo 1 2 3 0        c AMO: at most one true
h eo 4 5 6 0         c EO: exactly one true
h ek 2 1 2 3 0       c EK: exactly 2 of {x1,x2,x3} true
h d >=2 1 2 3 0      c CARD: at least 2 true

c --- CARD inequality variants ---
card 2 1 2 3 0       c >= 2 true (positive k, default >=)
card -3 1 2 3 4 0    c < 3 true (negative k, default <)
card >2 1 2 3 4 0    c > 2 true
card <=2 1 2 3 0     c <= 2 true

```

309 A.1 Automatic Simplification

310 The parser applies the following reductions:

- 311 ■ Unit EO and CNF constraints are extracted as unit-propagated prefix assignments.
- 312 ■ Unit AMO constraints are discarded (trivially satisfied).
- 313 ■ CARD-1 constraints are reduced to CNF; EK-1 to EO.
- 314 ■ CARD- n and EK- n (where k equals the number of literals) are converted to unit prefix assignments.
- 315 ■ CARD-0 constraints are discarded (trivially satisfied); EK-0 constraints yield negated prefix assignments.
- 316 ■ Conflicting unit literals, if detected, immediately raise an unsatisfiability error.